

CHƯƠNG 2

Hệ thống file & di chuyển

làm chủ `cd`, `pwd`, `ls`

“Trước khi học làm chủ một thành phố, bạn phải biết mình đang đứng ở góc phố nào, đường nào dẫn về nhà, và bản đồ tổng thể trông ra sao.”

Mục lục

1	Mục tiêu học tập	2
2	Cây thư mục là gì?	2
2.1	Ví von dễ hiểu nhất: cái tủ hồ sơ	2
2.2	Vì sao gọi là “cây”?	3
2.3	Ba khái niệm “vị trí” bạn phải thuộc	3
3	pwd — “Tôi đang đứng ở đâu?”	4
4	ls — Liệt kê nội dung	5
4.1	ls cơ bản	5
4.2	ls -l — xem chi tiết (dạng danh sách dọc)	5
4.3	ls -a — hiện cả file ẩn (dotfiles)	6
4.4	ls -la — kết hợp chi tiết + file ẩn	7
4.5	ls <đường_dẫn> — xem nội dung MỘT NƠI KHÁC mà không cần đi tới	7
5	cd — Di chuyển giữa các thư mục	8
5.1	Năm cách dùng cd cốt lõi	8
5.2	Thử ngay	9
6	Đường dẫn: tuyệt đối vs tương đối	10
6.1	Định nghĩa	10
6.2	Ba ký hiệu thần kỳ	10
6.3	Ví dụ SONG SONG: hai cách, cùng một đích	10
7	Các thư mục quan trọng trên hệ Unix/Linux	11
8	Lab chính — xây sân tập an toàn	13
9	Bảng đối chiếu macOS vs Linux (khía cạnh điều hướng)	16
10	Hiểu lầm thường gặp (đọc kỹ để không sa bẫy)	17
11	Tóm tắt 1 trang	18
12	Thẻ ghi nhớ (Flashcards)	19
13	Kế hoạch ôn ngắt quãng (Spaced Repetition)	19
14	Bài tập mở rộng (Stretch) — có đáp án	20
14.1	Bài 1 — Xây và khám phá cây sâu hơn	20
14.2	Bài 2 — Cùng đích, hai con đường	20
14.3	Bài 3 — Soi file ẩn và đếm	20
14.4	Bài 4 — “Quay đầu xe” liên tục	21
14.5	Bài 5 (tư duy Senior) — Bẫy đường dẫn tương đối	21
15	Đáp án phần “Tự kiểm tra”	22

Ở Chương 1, bạn đã biết Linux là gì, đã mở **Terminal** trên máy Mac và gõ vài lệnh đầu tiên như `uname`, `pwd`, `ls`. Bạn đã chạm tay vào “cánh cửa”. Chương này, chúng ta bước hẳn vào trong và học cách **đi lại** trong ngôi nhà đó mà không bị lạc.

Đừng lo nếu bạn chưa biết gì về máy tính. Mọi thuật ngữ sẽ được giải thích ngay khi xuất hiện. Bạn chỉ cần một thứ: **mở Terminal lên và gõ theo từng dòng**. Học bằng tay, không học bằng mắt.

1 Mục tiêu học tập

Học xong chương này, bạn sẽ có thể:

1. **Hiểu** “cây thư mục” (filesystem tree) là gì, và vẽ lại nó trong đầu như một cái tủ hồ sơ.
2. **Trả lời** được câu hỏi “tôi đang đứng ở đâu?” bằng lệnh `pwd`.
3. **Liệt kê** nội dung một thư mục với `ls`, và biết dùng các biến thể `ls -l`, `ls -a`, `ls -la`.
4. **Di chuyển** giữa các thư mục bằng `cd`: về nhà, lên thư mục cha, về chỗ cũ, về gốc.
5. **Phân biệt** đường dẫn tuyệt đối và đường dẫn tương đối — và biết khi nào dùng cái nào.
6. **Nhận diện** các thư mục quan trọng của hệ Unix/Linux (`/`, `/etc`, `/var`, `/tmp`, `/usr`, `/bin`...).
7. **Có phản xạ** dùng phím Tab để gõ nhanh và tránh sai chính tả.
8. **Bắt đầu suy nghĩ như một Senior**: vì sao đường dẫn tuyệt đối lại quan trọng sống còn trong automation.

Thời gian dự kiến: **45–60 phút** thực hành thật sự (không phải chỉ đọc).

2 Cây thư mục là gì?

2.1 Ví von dễ hiểu nhất: cái tủ hồ sơ

Hãy tưởng tượng một **tủ hồ sơ** khổng lồ trong văn phòng:

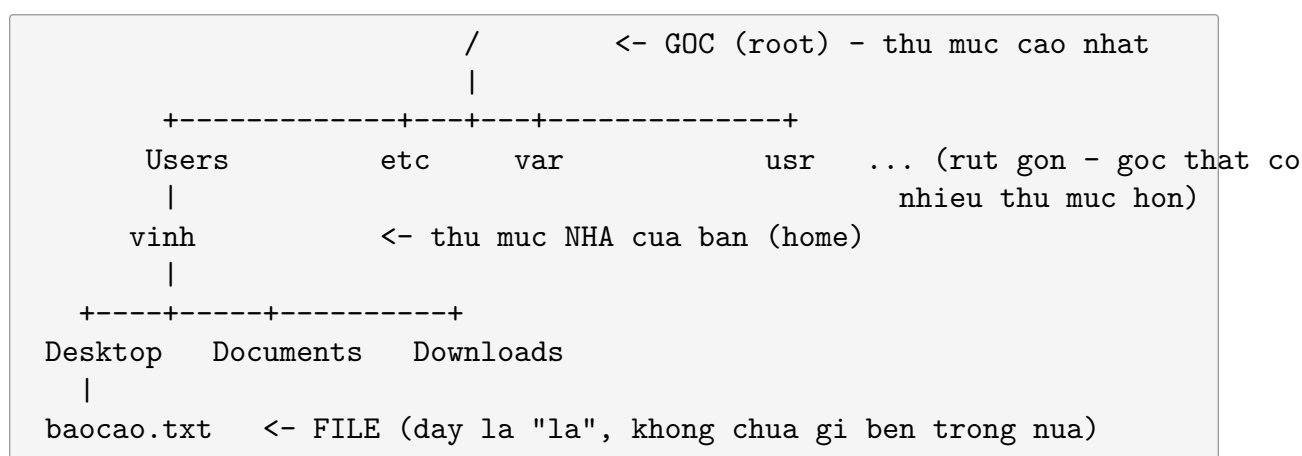
- Cái **tủ** lớn nhất, bao trùm tất cả → gọi là **thư mục gốc (root)**.
- Trong tủ có nhiều **ngăn kéo** → mỗi ngăn là một **thư mục (directory)**.
- Trong mỗi ngăn kéo lại có **cặp tài liệu nhỏ hơn** → các thư mục con, lồng nhau.
- Trong cùng là **tờ giấy** thật sự chứa thông tin → đó là **file (tệp)**.

Thuật ngữ

- **Thư mục (directory)**: một “ngăn chứa” có thể đựng các file và các thư mục khác. Trên giao diện đồ họa (như Finder của Mac), nó hiện ra dưới dạng cái **folder** (thư mục) màu xanh. “**Thư mục**” và “**folder**” là **MỘT THỨ**, chỉ khác cách gọi (sẽ nói kỹ ở phần hiểu lầm).
- **File (tệp)**: một mẫu dữ liệu cụ thể — một bức ảnh, một bài hát, một văn bản. Đây là “lá” của cây.

2.2 Vì sao gọi là “cây”?

Vì nếu vẽ ra, nó trông giống một cái cây lộn ngược (hoặc một cây gia phả):



Mọi thứ trên máy đều mọc ra từ **một gốc duy nhất**: dấu gạch chéo /. Không có gì nằm ngoài /. Đây là điểm khác biệt rất lớn so với Windows (nơi có ổ C:, ổ D: riêng biệt). Trong thế giới Unix/Linux/macOS, **chỉ có một cái cây duy nhất**, một gốc duy nhất.

Lưu ý cách đọc dấu /

Ký tự / đọc là “**slash**”. Khi nó đứng **MỘT MÌNH ở đầu** một đường dẫn, nó chỉ **thư mục gốc (root)**. Khi nó nằm **GIỮA** các tên (như trong /Users/vinh), nó chỉ là **dấu ngăn cách** giữa các tầng — không phải “gốc”. Đừng nhầm: chỉ cái / đầu tiên, đứng đơn độc, mới mang nghĩa “gốc”.

Sơ đồ trên đã **rút gọn** cho dễ nhìn. Trên thực tế, khi gõ `ls /` bạn sẽ thấy hàng chục mục chứ không chỉ bốn nhánh — điều đó hoàn toàn bình thường.

2.3 Ba khái niệm “vị trí” bạn phải thuộc

Khái niệm	Ý nghĩa	Ký hiệu tắt
Thư mục hiện tại (current / working directory)	Nơi bạn “đang đứng” lúc này trong cái cây	. (một dấu chấm)

Khái niệm	Ý nghĩa	Ký hiệu tắt
Thư mục cha (parent directory)	Thư mục chứa thư mục hiện tại (cao hơn một bậc)	.. (hai dấu chấm)
Thư mục nhà (home directory)	“Phòng riêng” của bạn — nơi mặc định chứa đồ của bạn	~ (dấu ngã)

Quan trọng — khác biệt macOS vs Linux

- Trên **macOS**, thư mục nhà của bạn nằm ở **/Users/<tên-bạn>**. Ví dụ **/Users/vinh**.
- Trên **Linux**, thư mục nhà thường nằm ở **/home/<tên-bạn>**. Ví dụ **/home/vinh**.
- Trên macOS **vẫn có** thư mục **/home**, nhưng nó thường **rỗng** (là một điểm gắn tự động — automount), **không** chứa nhà người dùng như trên Linux. Nếu bạn gõ **ls /home** trên Mac mà không thấy gì, đừng lo — hoàn toàn bình thường.
- Dấu **~** là cách viết tắt cho thư mục nhà, **bất kể** bạn đang ở hệ nào. Cứ gõ **~** là máy tự hiểu đúng nhà của bạn. Đây là một mẹo rất tiện.

Tự kiểm tra 1

Trên máy Mac, đường dẫn đầy đủ tới thư mục nhà của một người tên **lan** sẽ là gì? Còn trên Linux thì sao? (*Đáp án ở cuối chương.*)

3 pwd — “Tôi đang đứng ở đâu?”

pwd là viết tắt của **print working directory** — “in ra thư mục đang làm việc”. Đây là lệnh đầu tiên bạn nên gõ mỗi khi thấy hoang mang không biết mình ở đâu.

Thực hành ngay

Mở Terminal (nhấn **Cmd + Space**, gõ “Terminal”, Enter). Gõ:

```
pwd
```

Kết quả mong đợi (giả sử tên đăng nhập Mac của bạn là **vinh**):

```
/Users/vinh
```

Trấn an

Máy bạn sẽ hiển thị tên của **chính bạn**, không phải **vinh**. Nếu bạn thấy **/Users/lan** hay **/Users/john**, hoàn toàn bình thường — đó chính là tên tài khoản Mac của bạn. Khi vừa mở Terminal, bạn gần như luôn bắt đầu ở thư mục nhà.

Lệnh `pwd` không bao giờ thay đổi gì cả — nó chỉ **báo cáo vị trí**. Hoàn toàn an toàn, gõ bao nhiêu lần cũng được.

Tự kiểm tra 2

`pwd` là viết tắt của ba chữ gì? Lệnh này có làm thay đổi vị trí của bạn không?

4 `ls` — Liệt kê nội dung

`ls` viết tắt của `list` (liệt kê). Nó cho bạn thấy “trong ngăn kéo này có những gì”.

4.1 `ls` cơ bản

```
ls
```

Kết quả mong đợi (ví dụ — máy bạn chắc chắn khác):

Applications	Desktop	Documents	Downloads
Library	Movies	Music	Pictures
Public			

Trần an

Danh sách trên máy bạn sẽ khác hoàn toàn, tùy bạn đã lưu gì. Có thể nhiều hơn, có thể ít hơn. Không sao cả. Ngoài ra, **số cột và cách sắp xếp** trên màn hình bạn có thể khác (tùy độ rộng cửa sổ Terminal): có khi xếp thành nhiều cột, có khi mỗi mục một dòng — **cả hai đều đúng**. Đừng cố canh từng cột cho khớp với sách; chỉ cần để ý **các tên** xuất hiện.

4.2 `ls -l` — xem chi tiết (dạng danh sách dọc)

Chữ `-l` là một **tùy chọn** (option / flag) — một “công tắc” bật thêm tính năng. `-l` nghĩa là long (dài/chi tiết).

```
ls -l
```

Kết quả mong đợi (rút gọn):

```
total 0
drwx-----@  5 vinh  staff   160 Jun 20 09:14 Desktop
drwx-----+  4 vinh  staff   128 Jun 18 22:01 Documents
drwx-----+ 12 vinh  staff   384 Jun 25 14:33 Downloads
```

Mỗi dòng có nhiều cột. Bạn **chưa cần thuộc lòng** ngay bây giờ, chỉ cần làm quen sơ lược:

Cột (ví dụ)	Tên	Ý nghĩa sơ lược
drwx-----@	Quyền (permissions)	d đầu dòng = directory (thư mục). Phần rw x là quyền đọc/ghi/chạy. (<i>Học sâu ở chương về quyền.</i>)
5	Số liên kết (link count)	Một con số kỹ thuật; với thư mục, nó liên quan tới số thư mục con. Bạn chưa cần hiểu — sẽ học ở chương về liên kết/quyền.
vinh	Chủ sở hữu (owner)	Ai là chủ của mục này.
staff	Nhóm (group)	Nhóm người dùng sở hữu.
160	Kích thước (size)	Tính bằng byte.
Jun 20 09:14	Ngày sửa cuối	Lần chỉnh sửa gần nhất.
Desktop	Tên	Tên thư mục/file.

Ghi chú

Đừng hoảng vì cột quyền `drwx-----@`. Hiện tại chỉ cần nhớ: **chữ d ở đầu nghĩa là “đây là thư mục”**. Nếu là - (gạch ngang) ở đầu thì đó là một **file** thường.

4.3 `ls -a` — hiện cả file ẩn (dotfiles)

`-a` nghĩa là **all** (tất cả). Mặc định, `ls` **giấu** những file/thư mục có tên bắt đầu bằng dấu chấm `.` — gọi là **dotfile** (file ẩn).

Thuật ngữ — dotfile

Một file/thư mục mà tên bắt đầu bằng dấu chấm, ví dụ `.zshrc`, `.gitconfig`. Quy ước Unix coi đây là file **cấu hình/hệ thống**, người dùng thường không cần nhìn thấy hằng ngày, nên `ls` giấu chúng đi cho gọn. Chúng **không phải bị hỏng hay nguy hiểm** — chỉ là “ẩn”.

```
ls -a
```

Kết quả mong đợi (rút gọn):

.	.zsh_history	Desktop	Music
..	.zshrc	Documents	Pictures
.CFUserTextEncoding		Downloads	Public
.config	Library		

Trần an

Lại nhắc — số cột và cách dàn cột trên máy bạn có thể khác (đặc biệt vì có tên rất dài như `.CFUserTextEncoding`). Đừng cố canh từng cột; chỉ để ý các tên.

Để ý hai mục đặc biệt **luôn xuất hiện** với `ls -a`:

- `.` (một chấm) = **chính thư mục hiện tại**.
- `..` (hai chấm) = **thư mục cha**.

Hai mục này có mặt trong *mọi* thư mục và là chìa khóa để đi lại bằng đường dẫn tương đối (phần F).

4.4 `ls -la` — kết hợp chi tiết + file ẩn

Bạn có thể **gộp** nhiều tùy chọn lại. `ls -la` = vừa chi tiết (`-l`) vừa hiện hết (`-a`):

```
ls -la
```

`ls -la` và `ls -al` cho kết quả **giống hệt nhau** — thứ tự các chữ cái không quan trọng. Đây là lệnh nhiều DevOps gõ theo phản xạ hàng ngày.

4.5 `ls <đường_dẫn>` — xem nội dung MỘT NƠI KHÁC mà không cần đi tới

Bạn không cần `cd` vào một thư mục mới chỉ để liếc xem nó có gì. Đưa thẳng đường dẫn cho `ls`:

```
ls /Users
```

Kết quả mong đợi (ví dụ):

```
Shared  vinh
```

Hoặc xem thư mục `/etc` (chứa cấu hình hệ thống):

```
ls /etc
```

Bạn sẽ thấy nhiều file cấu hình. **Chỉ xem thôi, đừng sửa gì.**

Trấn an

Danh sách trong `/etc` trên máy bạn sẽ khác và có thể có những tên lạ (ví dụ thư mục do phần mềm bạn đã cài thêm tạo ra) — hoàn toàn bình thường. Chỉ cần xem, đừng sửa.

Tự kiểm tra 3

Bạn muốn vừa thấy file ẩn vừa thấy thông tin chi tiết của thư mục nhà. Gõ lệnh gì? Còn nếu chỉ muốn liệt kê nội dung của `/tmp` mà không rời khỏi chỗ đang đứng?

Góc Senior #1 — BSD `ls` (macOS) vs GNU `ls` (Linux)

Có một sự thật ít người mới biết: `ls` trên Mac **không hoàn toàn giống** `ls` trên Linux phổ biến (Ubuntu, CentOS...).

- macOS dùng phiên bản **BSD** (kế thừa từ dòng Unix của Berkeley).
- Đa số Linux dùng phiên bản **GNU** (gọi là *coreutils*).

Một hiểu lầm phổ biến là “cờ bật màu khác nhau”. Hãy nói cho chính xác:

- **Bật màu:** macOS hiện đại **hỗ trợ CẢ HAI** cách: `ls -G` (cách viết tắt cổ hữu của BSD) **lẫn** `ls --color=auto`. Nghĩa là `ls --color` **không** báo lỗi trên Mac đời mới — bạn cứ gõ thử sẽ thấy nó chạy. (Trên các bản macOS rất cũ thì `--color` có thể không có; nhưng máy đời nay thì có.)
- **Cảnh báo cái bẫy THẬT:** cùng cờ `-G` nhưng **mang ý nghĩa khác nhau** giữa hai họ! Trên **BSD/macOS**, `-G` = **bật màu**. Trên **GNU/Linux**, `-G` lại = **không in cột nhóm (group)** — hoàn toàn không liên quan tới màu. Đây mới chính là loại nhầm lẫn khiến script “chạy đúng kiểu này trên Mac, chạy ra kết quả khác trên Linux”.
- **Kích thước “dễ đọc”:** cả hai đều có `ls -lh` (`h` = **human-readable**, hiện 4.0K, 12M thay vì số byte trần) — cờ này giống nhau.
- Một số cờ dài kiểu `--option` của GNU (ví dụ `--time-style`, `--group-directories-first`, `--full-time`) **không tồn tại** trên BSD. Đây là nhóm cờ thực sự dễ làm “gãy” script khi chuyển từ Linux sang Mac (hoặc ngược lại).

Vì sao senior quan tâm? Vì bạn viết một script chạy ngon trên máy này, đẩy sang máy kia thì nó **gãy** hoặc **chạy ra kết quả khác** vì cờ không tồn tại hoặc mang nghĩa khác. Người mới thường mất hàng giờ debug chuyện này. Mẹo: nếu muốn `ls` trên Mac giống hệt Linux, có thể cài *coreutils* (qua Homebrew) để có `gls`. (Chưa cần làm bây giờ — chỉ cần biết “à, chúng khác nhau, và `-G` là cái bẫy”.)

5 cd — Di chuyển giữa các thư mục

`cd` viết tắt của **change directory** (đổi thư mục). Đây là lệnh “đi bộ” của bạn trong cái cây.

5.1 Năm cách dùng `cd` cốt lõi

Lệnh	Tác dụng	Ghi nhớ
<code>cd ~</code>	Về thư mục nhà	<code>~</code> = nhà
<code>cd ..</code>	Lên thư mục cha (cao hơn 1 bậc)	<code>..</code> = “đi lùi lên”
<code>cd -</code>	Về chỗ vừa ở trước đó	<code>-</code> = “quay đầu xe”
<code>cd /</code>	Về gốc (root, cao nhất)	<code>/</code> = gốc
<code>cd <đường_dẫn></code>	Đi tới một nơi cụ thể	Bạn chỉ định

Và một điều đặc biệt:

```
cd
```

`cd` **không kèm gì cả** cũng đưa bạn **về nhà** — y hệt `cd ~`. Đây là phím tắt rất hay dùng khi bạn bị “lạc” và muốn về điểm xuất phát.

Lưu ý quan trọng về `cd -`

Khác với mọi dạng `cd` khác (vốn “im lặng” khi thành công), riêng `cd -` sẽ **TỰ IN ra một dòng** cho biết nó vừa đưa bạn tới đâu (ví dụ in ra `/Users/vinh/linux-lab/tang1/tang2`). Đây là **hành vi bình thường**, không phải lỗi — đừng hoảng khi thấy dòng đường dẫn tự hiện ra.

5.2 Thử ngay

Thực hành ngay

```
cd /
pwd
```

Kết quả mong đợi:

```
/
```

Bạn vừa nhảy lên đỉnh cây. Giờ xem gốc có gì:

```
ls
```

Kết quả mong đợi (trên macOS, ví dụ — máy bạn có thể có nhiều mục hơn):

Applications	System	Users	bin
Library	Volumes	etc	usr
...			

Giờ về nhà:

```
cd
pwd
```

Kết quả mong đợi:

```
/Users/vinh
```

Trần an

Lại nhắc — chỗ `vinh` sẽ là tên bạn. Và danh sách ở `/` có thể nhiều mục lạ hoắc; bạn chưa cần hiểu hết. Quan trọng là bạn vừa **đi lên gốc rồi về nhà** thành công.

Tự kiểm tra 4

Bạn đang ở `/Users/vinh/Documents`. Gõ `cd ..` thì bạn tới đâu? Gõ tiếp `cd -` thì sao?

6 Đường dẫn: tuyệt đối vs tương đối

Đây là **khái niệm quan trọng nhất** của chương. Hiểu chắc phần này, bạn sẽ không bao giờ bị lạc.

6.1 Định nghĩa

- **Đường dẫn tuyệt đối (absolute path):** luôn bắt đầu bằng `/` (gốc). Nó mô tả đường đi từ gốc cây xuống đích. Giống địa chỉ đầy đủ: “Số 1, đường Lê Lợi, Quận 1, TP.HCM, Việt Nam” — ai đứng ở đâu cũng tìm ra đúng nơi.
 - Ví dụ: `/Users/vinh/Documents`
- **Đường dẫn tương đối (relative path):** không bắt đầu bằng `/`. Nó mô tả đường đi tính từ chỗ bạn đang đứng. Giống chỉ đường miệng: “đi thẳng 200m rồi rẽ phải” — chỉ đúng nếu bạn biết người đó đang đứng ở đâu.
 - Ví dụ: `Documents` (nghĩa là “thư mục Documents nằm ngay trong chỗ tôi đang đứng”)

6.2 Ba ký hiệu thần kỳ

Ký hiệu	Nghĩa
<code>.</code>	Thư mục hiện tại (“ở đây”)
<code>..</code>	Thư mục cha (lùi lên một bậc)
<code>~</code>	Thư mục nhà

6.3 Ví dụ SONG SONG: hai cách, cùng một đích

Giả sử bạn đang đứng ở thư mục nhà `/Users/vinh`, và muốn vào `/Users/vinh/Documents`.

Cách tuyệt đối (đi từ gốc, đứng ở đâu cũng đúng):

```
cd /Users/vinh/Documents
```

Cách tương đối (vì Documents nằm ngay trong chỗ tôi đứng):

```
cd Documents
```

Cả hai dẫn tới **cùng một nơi**. Kiểm chứng bằng `pwd` sau mỗi lần — kết quả giống hệt: `/Users/vinh/Documents`.

Một ví dụ song song nữa — giả sử bạn đang ở `/Users/vinh/Documents` và muốn sang `/Users/vinh/Downloads`:

```
cd /Users/vinh/Downloads      # tuyệt đối
```

```
cd ../Downloads               # tương đối: lui lên cha (ve /Users/vinh) rồi vào Downloads
```

Góc Senior #2 — Vì sao đường dẫn tuyệt đối quan trọng sống còn

Khi gõ tay trong Terminal, đường dẫn tương đối rất tiện (gõ ngắn). Nhưng trong **automation** — tức là khi máy tự chạy lệnh thay bạn — đường dẫn tương đối là **nguồn bug số một**.

- **Script** (file chứa danh sách lệnh để máy tự chạy) và **cron** (bộ hẹn giờ chạy lệnh tự động trên Linux) chạy trong một **môi trường tối giản** mà bạn **không ngờ tới**. Cron của một người dùng thường khởi động ở **thư mục nhà** của người đó — nhưng điều này không được bảo đảm tuyệt đối giữa các hệ.
- Nếu script viết `ls Documents` (tương đối), nó sẽ tìm `Documents` ở chỗ cron đang đứng — có thể **không tồn tại** → lỗi. Cùng script đó chạy tay trên máy bạn lại đúng. Loại bug “chạy tay thì được, chạy tự động thì hỏng” này tồn rất nhiều giờ debug.
- **Thủ phạm gây gây nhiều nhất** thực ra là **hai** thứ kết hợp: (1) thư mục hiện tại (cwd) **không đảm bảo**, và (2) **biến môi trường tối giản** của cron — đặc biệt `PATH` ngắn nên ngay cả tên lệnh cũng có thể không tìm thấy.
- **Quy tắc vàng của senior**: trong script và cron, **luôn dùng đường dẫn tuyệt đối** cho cả **file lẫn lệnh** (ví dụ `/bin/ls` thay vì `ls`), hoặc xác định rõ thư mục bằng `cd` ngay đầu script. Đừng bao giờ tin vào “thư mục hiện tại”.
- Đó cũng là lý do `pwd` hữu ích **trong script**: thêm một dòng `pwd` để in ra “tôi đang chạy ở đâu” giúp bạn debug cực nhanh khi mọi thứ đi sai.

7 Các thư mục quan trọng trên hệ Unix/Linux

macOS được xây trên nền Unix, nên nó **chia sẻ rất nhiều** thư mục chuẩn với Linux. Dưới đây là những thư mục bạn sẽ gặp đi gặp lại. Cứ `ls` để xem, nhưng **không sửa/xóa** gì trong vùng hệ thống.

Thư mục	Tên gọi	Chứa gì	Trên macOS?
<code>/</code>	root (gốc)	Gốc của mọi thứ. Mọi đường dẫn tuyệt đối bắt đầu từ đây.	Có
<code>/Users</code>	(macOS)	Thư mục nhà của các người dùng. Tương đương <code>/home</code> của Linux.	Có (đây là chỗ của bạn)

Thư mục	Tên gọi	Chứa gì	Trên macOS?
/home	(Linux)	Thư mục nhà người dùng trên Linux.	Có tồn tại nhưng thường rỗng (automount); nhà của bạn ở /Users
/etc	et-cetera	Cấu hình toàn hệ thống (settings). Ví dụ cấu hình mạng, người dùng.	Có
/var	variable	Dữ liệu thay đổi liên tục : log (nhật ký), hàng đợi, cache.	Có
/tmp	temporary	File tạm , có thể bị xóa khi khởi động lại. Vùng “nháp”.	Có
/usr	Unix System Resources	Chương trình & thư viện cài cho người dùng (không phải file cá nhân của bạn).	Có
/bin	binaries	Một nhóm lệnh/chương trình lõi tối thiểu (như ls, cp, mv, rm, cat...).	Có

Lưu ý về /bin và shell builtin

/bin chứa các **file chương trình** thật sự như ls, cp, mv, rm, cat. Nhưng **cd KHÔNG nằm ở đây** — cd là một **lệnh tích hợp sẵn trong shell (shell builtin)**, không phải file thực thi riêng. Nếu bạn gõ ls /bin/cd sẽ thấy báo No such file or directory, và type cd sẽ trả lời cd is a shell builtin. (pwd cũng có dạng builtin tương tự.) Ngoài ra, nhiều lệnh khác nằm ở /usr/bin chứ không phải /bin — /bin chỉ giữ một nhóm lệnh lõi tối thiểu.

Lưu ý macOS

Apple đặt thêm vài thư mục riêng (/System, /Applications, /Library, /Volumes). Và macOS hiện đại **bảo vệ rất chặt** vùng hệ thống (cơ chế SIP — System Integrity Protection), nên kể cả muốn nghịch đại cũng khó. Dù vậy: **thói quen tốt là không đụng vào thư mục hệ thống khi chưa hiểu rõ.**

Góc Senior #3 — FHS (Filesystem Hierarchy Standard)

Bố cục thư mục ở trên không phải ngẫu nhiên. Nó tuân theo một **tiêu chuẩn** tên là **FHS — Filesystem Hierarchy Standard** (Tiêu chuẩn phân cấp hệ thống file). FHS quy định “loại file gì thì nằm ở đâu” trên các hệ Linux. Vì sao senior **bắt buộc** phải thuộc bố cục này?

- Khi vào một server Linux lạ hoặc lúc 2 giờ sáng để xử lý sự cố, bạn phải biết

log nằm ở /var/log, cấu hình nằm ở /etc — không có thời gian mò mẫm.

- Hỏng dịch vụ web? → đọc /var/log. Sửa cấu hình nginx? → vào /etc/nginx. File tạm đầy ổ đĩa? → kiểm tra /tmp và /var.
- FHS giúp **mọi máy Linux trông giống nhau** ở những chỗ quan trọng, nên kỹ năng của bạn **chuyển được** từ máy này sang máy khác.

macOS **không tuân thủ FHS 100%** (Apple có cấu trúc riêng cho phần đồ họa và ứng dụng), nhưng phần “lõi Unix” thì rất giống. Học trên Mac → áp dụng được phần lớn sang Linux server.

Tự kiểm tra 5

Trên một server Linux, bạn muốn xem nhật ký (log) của hệ thống thì thường vào thư mục nào? Còn cấu hình hệ thống nằm ở đâu?

8 Lab chính — xây sân tập an toàn

Giờ là phần quan trọng nhất: **gỡ tay toàn bộ**. Lab này **tuyệt đối an toàn** — chúng ta chỉ điều hướng và tạo/xóa thư mục **tập rỗng**, không đụng tới file thật của bạn.

Bước 1 — Xác định vị trí

```
pwd
```

Kết quả mong đợi: /Users/vinh (tên bạn thay cho vinh). Nếu không phải, gỡ cd để về nhà trước.

Bước 2 — Xem có gì trong nhà

```
ls -la
```

Kết quả mong đợi: một danh sách dài, có cả file ẩn (.zshrc, .config...) lẫn thư mục thường (Desktop, Documents...). Bạn thấy . và .. ở trên cùng.

Thuật ngữ — mkdir

Viết tắt **make directory** (tạo thư mục). Cờ **-p** (= **p**arents) bảo nó **tạo luôn cả các thư mục cha** nếu chưa có, một mạch nhiều tầng. Không có -p, mkdir sẽ báo lỗi nếu thư mục cha chưa tồn tại.

Bước 3 — Tạo sân tập (giới thiệu mkdir)

```
mkdir -p ~/linux-lab/tang1/tang2
```

Lệnh này tạo một mạch ba tầng: **linux-lab** → bên trong có **tang1** → bên trong có **tang2**. Lệnh chạy xong **không in ra gì** — trong Unix, “im lặng nghĩa là thành công”.

Kiểm tra:

```
ls ~/linux-lab
```

Kết quả mong đợi:

```
tang1
```

Bước 4 — Đi vào tận đáy bằng đường dẫn tuyệt đối

```
cd ~/linux-lab/tang1/tang2
pwd
```

Kết quả mong đợi:

```
/Users/vinh/linux-lab/tang1/tang2
```

Bạn đang đứng ở tầng sâu nhất.

Bước 5 — Đi lại bằng đường dẫn TƯƠNG ĐỐI

Lên thư mục cha (**tang1**):

```
cd ..
pwd
```

Kết quả mong đợi: /Users/vinh/linux-lab/tang1

Giờ thử lên **hai bậc một lúc**. Để rõ ràng, ta quay về đáy trước cho chắc rồi mới lên hai bậc:

```
cd ~/linux-lab/tang1/tang2      # về lại đáy cho chắc
cd ../../..                     # lên 2 bậc
pwd
```

Kết quả mong đợi: /Users/vinh/linux-lab

(Từ **tang2** lên 2 bậc: **tang2** → **tang1** → **linux-lab**.)

Giờ thử “quay đầu xe” với **cd -**:

```
cd -
pwd
```

Kết quả mong đợi: /Users/vinh/linux-lab/tang1/tang2

Ghi chú

Ngay khi bạn gõ `cd -`, nó sẽ **tự in ra một dòng** đường dẫn (chính là `/Users/vinh/linux-lab/tang1/tang2`) — đây là hành vi bình thường, riêng có của `cd -`. Sau đó `pwd` xác nhận lại vị trí.

`cd -` đưa bạn về **chỗ ngay trước đó** (chính là `tang2`). Lệnh này cực tiện khi bạn nhảy qua lại giữa hai thư mục xa nhau. Gõ `cd -` lần nữa sẽ lại quay về `linux-lab` — nó như nút “qua lại” giữa hai vị trí gần nhất.

Bước 6 — Mẹo gõ nhanh: phím Tab (tab-completion)

Đây là **kỹ năng đổi đời** cho người mới. Thử:

1. Gõ: `cd ~/linux-l` rồi **DỪNG**, nhấn phím **Tab**.
2. Terminal **tự hoàn thành** thành `cd ~/linux-lab/`.
3. Nhấn **Tab** tiếp → tự thêm `tang1/` (vì đó là lựa chọn duy nhất).
4. Nhấn **Tab** tiếp → tự thêm `tang2/`.

Tab-completion = gõ vài chữ đầu rồi nhấn Tab để máy điền nốt phần còn lại của tên thư mục/file. Lợi ích:

- **Nhanh hơn** nhiều, đỡ mỏi tay.
- **Tránh gõ sai chính tả** — nếu Tab không hoàn thành, nghĩa là tên đó **không tồn tại** (báo lỗi sớm!).
- Nếu có nhiều lựa chọn trùng đầu, nhấn Tab hai lần để **liệt kê** tất cả khả năng.

Thuật ngữ — `rmdir`

Viết tắt **remove directory** (xóa thư mục). Điểm cực kỳ quan trọng: **`rmdir` CHỈ xóa được thư mục RỖNG**. Nếu thư mục còn chứa bất cứ thứ gì, nó sẽ **từ chối** và báo lỗi. Chính vì thế `rmdir` rất an toàn cho người mới — nó **không thể** vô tình xóa nhầm dữ liệu của bạn.

Bước 7 — Thử “lưới an toàn” của `rmdir` **TRƯỚC** khi dọn

Trước khi dọn sạch, hãy **chứng kiến tận mắt** lưới an toàn này. Lúc này cây `~/linux-lab/tang1/tang2` vẫn còn nguyên, nên `linux-lab` đang chứa `tang1`. Thử xóa `linux-lab` khi nó **chưa rỗng**:

```
rmdir ~/linux-lab
```

Kết quả mong đợi:

```
rmdir: /Users/vinh/linux-lab: Directory not empty
```


rmdir từ chối vì thư mục còn nội dung bên trong. Đây chính là “lưới an toàn” — **rmdir** không thể vô tình xóa dữ liệu của bạn.

Bước 8 — Dọn sân tập đúng cách (xóa từ trong ra ngoài)

Trước hết về nhà cho an toàn (không xóa thư mục mình đang đứng trong đó):

```
cd ~
```

Vì phải **rỗng** mới xóa được, ta xóa từ tầng sâu nhất (**tang2**) ra ngoài dần:

```
rmdir ~/linux-lab/tang1/tang2
rmdir ~/linux-lab/tang1
rmdir ~/linux-lab
```

Mỗi lệnh thành công sẽ **im lặng**. Kiểm tra đã xóa sạch:

```
ls ~/linux-lab
```

Kết quả mong đợi:

```
ls: /Users/vinh/linux-lab: No such file or directory
```

Báo “No such file or directory” (không có file/thư mục này) — nghĩa là dọn sạch thành công!

Cảnh báo

Việc xóa thư mục **có nội dung** (và xóa file) cần lệnh khác (**rm**) — một lệnh **nguy hiểm** mà ta sẽ học cẩn thận ở chương sau.

Tự kiểm tra 6

Vì sao **rmdir** được coi là an toàn cho người mới? Nếu thư mục còn file bên trong, **rmdir** sẽ làm gì?

9 Bảng đối chiếu macOS vs Linux (khía cạnh điều hướng)

Khía cạnh	macOS	Linux	Ghi chú
Thư mục nhà thực sự	/Users/<tên>	/home/<tên>	Khác biệt rõ rệt nhất; nhớ dùng ~ để khỏi lo
/home	Tồn tại nhưng thường rỗng (automount)	Là nơi chứa nhà người dùng	Trên Mac, <code>ls /home</code> có thể không thấy gì — bình thường

Khía cạnh	macOS	Linux	Ghi chú
<code>pwd</code> , <code>cd</code> , <code>ls</code> (cơ bản)	Giống	Giống	Họ Unix chung
<code>~</code> , <code>.</code> , <code>..</code> , <code>/</code>	Giống	Giống	Ký hiệu đường dẫn như nhau
<code>mkdir -p</code> , <code>rmdir</code>	Giống	Giống	Hành xử như nhau
Phiên bản <code>ls</code>	BSD	GNU (coreutils)	Cả hai đều bật màu được, nhưng <code>-G</code> mang nghĩa KHÁC NHAU (BSD: bật màu; GNU: ẩn cột group); một số cờ dài <code>--option</code> của GNU không có trên BSD
<code>/etc</code> , <code>/var</code> , <code>/tmp</code> , <code>/usr</code> , <code>/bin</code>	Có	Có	Phần lõi Unix giống nhau
Tuân thủ FHS	Một phần	Đầy đủ hơn	Apple có cấu trúc riêng cho GUI/app

Kết luận quan trọng: 90% những gì bạn học về điều hướng trên Mac **áp dụng thẳng** sang Linux server. Đây là lý do học DevOps trên Mac rất thuận lợi.

10 Hiểu lầm thường gặp (đọc kỹ để không sa bẫy)

1. “Thư mục và folder là hai thứ khác nhau.”
→ **Sai**. Chúng là **một**. “Folder” là tên gọi trong giao diện đồ họa (Finder), “directory” là tên gọi trong dòng lệnh. Cùng một vật.
2. “Phải nhớ thuộc lòng mọi đường dẫn.”
→ **Không cần**. Đã có **phím Tab** điền hộ, có `pwd` để biết mình ở đâu, có `ls` để nhìn xung quanh. Senior không nhớ đường dẫn — họ **điều hướng và dùng Tab**.
3. “`cd` bắt buộc phải dùng đường dẫn tuyệt đối.”
→ **Sai**. `cd` nhận cả tương đối (`cd Documents`, `cd ..`) lẫn tuyệt đối. Khi gõ tay, tương đối thường nhanh hơn.
4. “Lệnh chạy xong không in gì là bị lỗi.”
→ **Sai**. Trong Unix, **im lặng = thành công**. `mkdir`, `rmdir` và **hầu hết** các dạng `cd` thành công đều không in gì. **Ngoại lệ:** `cd -` cố ý in ra nơi nó vừa chuyển tới. Có thông báo đở/lỗi mới là chuyện đáng lo.
5. “File ẩn (dotfile) là file hỏng hoặc virus.”
→ **Sai**. Dotfile chỉ là file cấu hình được **giấu cho gọn**. `.zshrc`, `.gitconfig` là bạn thân của DevOps.

6. “~ và / giống nhau.”

→ **Sai.** / là gốc của cả cây (cao nhất). ~ là **nhà của bạn** (/Users/<tên>), nằm sâu bên trong cây.

7. “cd là một chương trình nằm trong /bin.”

→ **Sai.** cd là **lệnh tích hợp sẵn của shell (builtin)**, không phải file thực thi. ls /bin/cd sẽ báo “No such file or directory”. Còn ls, cp, mv, rm thì mới là file thật trong /bin.

11 Tóm tắt 1 trang

Tóm tắt

Cây thư mục: mọi thứ mọc từ gốc /. Thư mục lồng nhau như tử hờ sơ; file là “lá”.

Ba vị trí: . = đây | .. = cha | ~ = nhà (macOS: /Users/<tên>, Linux: /home/<tên>).

Lệnh lỗi:

Lệnh	Tác dụng
pwd	Tôi đang ở đâu (in đường dẫn tuyệt đối)
ls	Liệt kê nội dung
ls -l	Chi tiết (quyền, chủ, size, ngày)
ls -a	Hiện cả file ẩn (dotfile)
ls -la	Chi tiết + ẩn
ls <path>	Xem nơi khác mà không cần đi tới
cd <path>	Đi tới
cd hoặc cd ~	Về nhà
cd ..	Lên cha
cd -	Về chỗ trước (lệnh này TỰ IN đường dẫn)
cd /	Về gốc
mkdir -p a/b/c	Tạo cây thư mục nhiều tầng
rmdir <dir>	Xóa thư mục RỎNG (an toàn)

Đường dẫn: tuyệt đối bắt đầu bằng / (đúng ở mọi nơi) | tương đối tính từ chỗ đang đứng.

Phản xạ vàng: dùng **Tab** để điền tên; dùng **đường dẫn tuyệt đối trong script/cron** (cho cả file lẫn lệnh).

Thư mục cần biết: /etc (cấu hình), /var/log (nhật ký), /tmp (tạm), /usr (chương trình), /bin (lệnh lỗi). Lưu ý: cd/pwd là shell builtin, không nằm trong /bin.

12 Thẻ ghi nhớ (Flashcards)

Che cột phải, tự trả lời, rồi mở ra kiểm tra.

Mặt trước (hỏi)	Mặt sau (đáp)
<code>pwd</code> làm gì?	In thư mục hiện tại (print working directory)
<code>ls -a</code> hiện thêm gì?	File ẩn (dotfile, tên bắt đầu bằng <code>.</code>)
<code>ls -l</code> cho biết gì?	Chi tiết: quyền, chủ sở hữu, kích thước, ngày sửa
<code>cd ..</code> đi đâu?	Lên thư mục cha (1 bậc)
<code>cd -</code> đi đâu?	Về thư mục vừa ở trước đó (và TỰ IN đường dẫn đó ra)
<code>cd</code> (không tham số) đi đâu?	Về thư mục nhà
Ký hiệu <code>~</code> nghĩa là?	Thư mục nhà (macOS: <code>/Users/<tên></code>)
Ký hiệu <code>.</code> và <code>..</code> ?	<code>.</code> = thư mục hiện tại; <code>..</code> = thư mục cha
Đường dẫn tuyệt đối bắt đầu bằng?	Dấu <code>/</code> (gốc)
<code>mkdir -p</code> khác <code>mkdir</code> ở đâu?	<code>-p</code> tạo luôn các thư mục cha còn thiếu
<code>rmdir</code> chỉ xóa được gì?	Thư mục RỖNG (an toàn)
Thư mục log trên Linux?	<code>/var/log</code>
Thư mục cấu hình hệ thống?	<code>/etc</code>
FHS là gì?	Filesystem Hierarchy Standard — chuẩn “file gì nằm ở đâu”
<code>cd</code> là chương trình trong <code>/bin</code> ?	KHÔNG — <code>cd</code> là shell builtin, không phải file thực thi
Bật màu cho <code>ls</code> : macOS vs Linux?	Cả hai đều có <code>ls --color</code> ; macOS có thêm <code>ls -G</code> . Cảnh thận: <code>-G</code> trên GNU/Linux lại mang nghĩa KHÁC (ẩn cột group, không phải màu)

13 Kế hoạch ôn ngắt quãng (Spaced Repetition)

Ghi nhớ lâu dài cần **ôn lại đúng lúc sắp quên**. Theo lịch:

- **Sau 1 ngày:** Mở Terminal, KHÔNG nhìn sách, làm lại Lab Chính từ đầu (tạo `~/linux-lab/tang1/tang2`, đi lại tương đối, dọn bằng `rmdir`). Đọc lại “Tóm tắt 1 trang”.
- **Sau 3 ngày:** Lật bộ Thẻ ghi nhớ, tự trả lời. Làm Bài tập mở rộng #2 và #3 (phần N). Tự giải thích cho mình (hoặc cho người khác) sự khác nhau giữa đường dẫn tuyệt đối và tương đối.

- **Sau 1 tuần:** Làm toàn bộ “Tự kiểm tra” 1→6 mà không xem đáp án trước. Làm lại Bài tập mở rộng #1. Nếu trả lời trôi chảy $\geq 80\%$ → bạn đã nắm chắc chương này, sẵn sàng sang chương sau.

Mẹo senior

Sau mỗi buổi, **xóa lịch sử trong đầu** rồi gõ lại bằng phản xạ. Cơ bắp ngón tay nhớ lệnh còn quan trọng hơn trí nhớ lý thuyết.

14 Bài tập mở rộng (Stretch) — có đáp án

14.1 Bài 1 — Xây và khám phá cây sâu hơn

Tạo cấu trúc `~/duan/web/src/components`, đi vào tận `components`, in vị trí, rồi quay về nhà chỉ bằng **MỘT** lệnh.

Đáp án / kết quả mong đợi

```
mkdir -p ~/duan/web/src/components
cd ~/duan/web/src/components
pwd          # -> /Users/<ten>/duan/web/src/components
cd           # ve nha bang mot lenh (hoac cd ~)
pwd          # -> /Users/<ten>
```

Dọn dẹp: `rmdir ~/duan/web/src/components ~/duan/web/src ~/duan/web`
`~/duan` (xóa từ trong ra).

14.2 Bài 2 — Cùng đích, hai con đường

Bạn đang ở `~/duan/web`. Hãy vào `~/duan` bằng **hai cách**: một tuyệt đối, một tương đối.

Đáp án

```
cd ~/duan/web
cd /Users/<ten>/duan    # tuyet doi (thay <ten> bang ten ban)
# hoac:
cd ~/duan/web
cd ..                  # tuong doi - lui len cha
```

Cả hai đều cho `pwd` → `/Users/<tên>/duan`.

14.3 Bài 3 — Soi file ẩn và đếm

Hiển thị **tất cả** nội dung thư mục nhà ở dạng chi tiết, và tìm xem có file `.zshrc` không.

Đáp án / kết quả mong đợi

```
ls -la ~
```

Tìm dòng có tên `.zshrc` ở cuối. Nếu thấy một dòng kết thúc bằng `.zshrc` → có. Nếu máy bạn dùng shell khác có thể không có `.zshrc` mà có `.bash_profile` — cả hai đều là dotfile cấu hình, không sao cả.

(Nâng cao — xem trước chương sau): `ls -la ~ | grep zshrc` sẽ lọc chỉ in dòng chứa “zshrc”. `grep` là lệnh tìm kiếm, ta học sau.

14.4 Bài 4 — “Quay đầu xe” liên tục

Đứng ở nhà, `cd /etc`, rồi `cd /var`, rồi dùng `cd -` hai lần. Dự đoán bạn ở đâu sau mỗi lần trước khi gõ. (Nhớ: mỗi lần `cd -` sẽ tự in ra đường dẫn nó vừa chuyển tới.)

Đáp án

```
cd ~          # o /Users/<ten>
cd /etc       # o /etc   (truoc do: nha)
cd /var       # o /var   (truoc do: /etc)
cd -          # -> /etc  (quay ve cho truoc = /etc; lenh in ra dong /etc)
cd -          # -> /var  (lai quay ve cho truoc = /var; lenh in ra dong /var)
```

`cd -` luôn nhảy qua lại giữa **hai** vị trí gần nhất, và **tự in ra** đường dẫn đích mỗi lần.

14.5 Bài 5 (tư duy Senior) — Bẫy đường dẫn tương đối

Một bạn viết script chứa dòng `ls duan`. Chạy tay trong thư mục nhà thì OK, nhưng đặt vào cron lại báo `No such file or directory`. Vì sao, và sửa thế nào?

Đáp án

Vì sao: `duan` là đường dẫn **tương đối**, tính từ “thư mục hiện tại”. Khi chạy tay, thư mục hiện tại là nhà (`/Users/<tên>`) nên thấy `duan`. Cron của user thường khởi động ở thư mục **nhà** của user — nhưng điều khiến script gây nhiễu nhất là: (1) thư mục hiện tại **không được bảo đảm**, và (2) cron chạy trong **môi trường tối giản** (PATH ngắn, thiếu biến môi trường), nên ngay cả tên lệnh cũng có thể không tìm thấy.

Sửa: Dùng **đường dẫn tuyệt đối** cho cả file lẫn lệnh:

```
/bin/ls /Users/<ten>/duan
```

Hoặc xác định thư mục ngay đầu script: `cd /Users/<tên>` trước khi chạy các lệnh tương đối. Đây chính là **Quy tắc vàng** ở Góc Senior #2.

15 Đáp án phần “Tự kiểm tra”

1. macOS: `/Users/lan`. Linux: `/home/lan`. (Dấu `~` đại diện cho cả hai tùy hệ.)
2. `pwd` = **p**rint **w**orking **d**irectory. **Không** làm thay đổi vị trí — chỉ in ra.
3. Xem chi tiết + ẩn của nhà: `ls -la ~` (hoặc `ls -la` khi đang ở nhà). Liệt kê `/tmp` mà không rời chỗ: `ls /tmp`.
4. `cd ..` từ `/Users/vinh/Documents` → tới `/Users/vinh`. Gõ tiếp `cd -` → quay lại `/Users/vinh/Documents` (chỗ vừa rời), và lệnh sẽ tự in ra dòng đường dẫn đó.
5. Log: `/var/log`. Cấu hình hệ thống: `/etc`.
6. `rmdir` an toàn vì nó **chỉ xóa được thư mục RỖNG** — không thể vô tình xóa dữ liệu. Nếu thư mục còn file bên trong, `rmdir` **từ chối** và báo `Directory not empty`.

Chúc mừng! Bạn vừa làm chủ kỹ năng đi lại trong hệ thống file — nền tảng cho mọi việc DevOps sau này. Mọi server bạn sẽ quản lý đều bắt đầu bằng `pwd`, `ls`, `cd`. Ở **Chương 3**, chúng ta sẽ học **tạo, xem, sao chép, di chuyển và xóa file** (gặp lại lệnh `rm` đầy quyền lực và cách dùng nó mà không “tự bắn vào chân”). Hẹn gặp lại!